structure.md — Genetic Swarm Business Operating System

Version 2.1 · Research-Integrated Edition · July 2026
Implementation mapping: §12 · SDD pack: `planning/structure/` · Status: `status.md`

0. What This Is

A governed, self-improving multi-agent system that:

1. Learns how a business actually operates (from documents, experts, AND real event logs).
2. Distills that knowledge into reusable rules, skills, workflows, and playbooks.
3. Executes work through bounded, auditable agent workflows.
4. Evolves those workflows in a sandbox — never directly in production.

Design priorities, in order: **Safety → Auditability → Correctness → Efficiency → Autonomy.** Autonomy is earned per workflow through evidence, not granted by default.

This document is the **architecture vision and source of truth**. Executable decomposition (requirements / design / tasks), product-bar evidence, and as-built notes live in §12 and the linked planning files — they refine this document; they do not replace it.

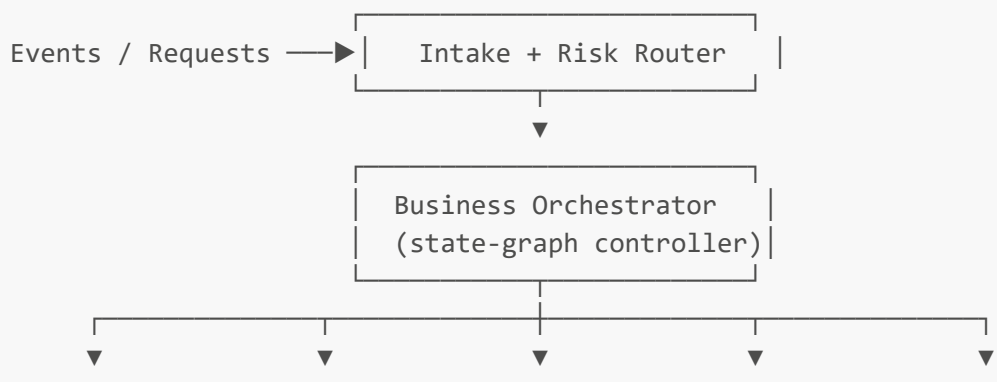
---

1. Core Principles

1. **Evidence over opinion.** Learn from real traces, not only from what people say they do.
2. **Bounded autonomy.** Every action has a risk tier, a permission scope, and (where needed) a human gate.
3. **Everything is testable.** No agent, prompt, or workflow reaches production without passing an eval.
4. **Sandbox evolution.** The evolution engine proposes; it never mutates production directly.
5. **Provenance always.** Every rule, decision, and memory traces back to a source.
6. **Reversibility first.** Prefer reversible actions; require rollback plans for the rest.
7. **Human-centered.** Show confidence, show evidence, make correction easy.

---

2. High-Level Architecture



```
    Process        Knowledge      Execution     Governance     Security
    Intelligence   Layer          Workflows     Officer        Red-Team
    Layer          (RAG+Graph+    (bounded       (risk tiers,   (adversarial
    (event logs)   Memory)        agents)        approvals)     testing)
         └──────────────┴──────────────┼──────────────┴──────────────┘
                                       ▼
                            Audit Log + Memory Write
                                       ▼
                            Evaluation + Human Review
                                       ▼
                     Evolution Sandbox (propose → test → canary)
```

Six layers work together: **Process Intelligence** (2.3), **Knowledge** (3), **Execution** (4), **Evolution** (5), **Governance** (6), and **Security** (7), all wrapped by **Evaluation** (8).

2.3 Process Intelligence Layer

The swarm must not learn only from documents and interviews. It should learn from **actual operational traces**: tickets, CRM/ERP actions, calendar events, emails, approvals, file edits, API calls, and completion records. Process mining turns these logs into discoverable workflow models, conformance checks, and bottleneck analysis — the empirical version of your "Shadow Mode."

New agents:

- **Process Miner Agent** — discovers real workflows from event logs.
- **Task Mining Agent** — observes UI/human-level steps where permitted.
- **Conformance Agent** — compares actual work against documented SOPs.
- **Bottleneck Analyzer** — finds delays, loops, rework, and handoff failures.
- **Causal Improvement Agent** — proposes interventions likely to improve outcomes.

Event Log Schema:

```yaml
event:
  id: "evt_..."
  timestamp: "2026-07-06T14:03:00Z"
  actor_type: "human | agent | system"
  actor_id: "user_or_agent_id"
  process_id: "customer_onboarding"
  case_id: "customer_12345"
  activity: "review_contract"
  input_refs: ["doc_contract_v3"]
  output_refs: ["approval_decision_789"]
  tools_used: ["crm", "email"]
  decision_point: true
  decision_reason_summary: "Contract had non-standard liability clause."
  confidence: 0.82
  risk_tier: "tier_3_execute_reversible"
  human_approved: true
  outcome:
    status: "completed"
```

```
    latency_minutes: 42
    quality_score: 0.94
```

---

3. Knowledge Acquisition, Distillation & Memory

3.1 Elicitation Methods

Expert knowledge is largely tacit — which cues matter, when to override the rule, when something "feels wrong." Cognitive Task Analysis and the Critical Decision Method are the established ways to surface this. Upgrade the single "Expert Shadow Agent" into a multi-method capture system.

Method	Best For	Output
Shadow Mode	Real actions	Event logs, action traces
Critical Decision Interview	Rare / high-stakes calls	Decision requirement cards
Think-Aloud Session	Routine expert work	Step-by-step heuristics
Exception Interview	Edge cases	Exception library
Retrospective Review	Completed cases	Lessons learned
Apprentice Mode	Expert teaches swarm	Skills, playbooks

3.2 Decision Requirement Card

```
decision_requirement:
  id: "drc_contract_exception_001"
  domain: "legal_operations"
  decision_point: "approve_non_standard_clause"
  expert_sources: ["senior_counsel_A", "contract_manager_B"]
  context_signals: ["customer_size", "liability_cap", "jurisdiction",
"renewal_value"]
  cues_experts_notice:
    - "Clause shifts uncapped indirect damages to company."
    - "Customer insists on governing law outside approved list."
  normal_action: "route_to_legal_review"
  exception_paths:
    - condition: "enterprise_customer AND pre-approved fallback accepted"
      action: "approve_with_note"
  red_flags: ["unlimited liability", "data protection indemnity"]
  required_evidence: ["contract_diff", "customer_risk_profile",
"approval_history"]
  risk_tier: "tier_4_execute_with_gate"
  human_approval_required: true
  validation_tests:
    - "Does recommendation match senior counsel decision on historical cases?"
  confidence: 0.78
  last_reviewed: "2026-07-06"
```

3.3 Hybrid Memory

One generic "knowledge base" is not enough. Long-running agents need differentiated memory — raw observations, higher-level reflections, and retrievable long-term stores (a pattern proven in work like Generative Agents and hierarchical-memory agent designs).

Memory Type	Stores	Example
Event	Raw operational logs	"Agent sent invoice at 9:42 AM."
Episodic	Case narratives	"This renewal almost failed — legal was pulled in late."
Semantic	Facts / rules	"Enterprise contracts over 250k need legal review."
Procedural	Skills / workflows	"How to onboard a new client."
Decision	Decisions + reasons	"We approved exception X because Y."
Exception	Edge cases	"If supplier in region Z, use alternate process."
Evaluation	Test results	"Workflow v12 failed privacy test."
Provenance	Source attribution	"Rule came from SOP v4 and expert Alice."

3.4 Retrieval: Tiered Hybrid (LightRAG core)

GraphRAG-style community summarization is deliberately avoided — its per-chunk extraction plus community-report generation makes both initial indexing and (critically) re-indexing on every new document prohibitively expensive for a system that ingests event logs and documents continuously.

Instead, use a cost-tiered retrieval stack. Most queries never touch the expensive tiers.

Tier 0 — Vector search (default, cheapest). Semantic similarity over chunked documents. Handles the majority of "find the relevant passage" queries. No graph, no extra LLM calls.

Tier 1 — LightRAG graph layer (relational reasoning).

- Graph-based text index with dual-level retrieval: low-level (entity-specific) + high-level (thematic).
- **Incremental updates** — new documents/events are added without rebuilding the graph. This is the primary reason for choosing LightRAG.
- Runs well against self-hosted / local models to cut token cost further.
- Answers relational questions: "which obligations depend on this contract?", "who touched this case and in what order?"

Tier 2 — Hierarchical summaries (RAPTOR-style, optional, on demand). Recursive cluster-and-summarize tree, built only for corpora that get frequent corpus-wide questions ("recurring root causes across failed onboarding cases"). Built lazily, not for the whole knowledge base.

Always-on — Provenance layer. Every answer cites its source documents, experts, event logs, or decisions, regardless of which tier served it.

Escalation rule: start at Tier 0 → escalate to Tier 1 only when the query needs relationships/multi-hop → escalate to Tier 2 only for global synthesis. This keeps 80%+ of traffic on the cheapest tier.

Off-the-shelf option: If a build is not desired, AnythingLLM or RAGFlow provide open-source, local-model-friendly RAG platforms; LightRAG can be integrated as the graph layer behind them.

Evaluation: score retrieval separately on context relevance, answer relevance, and faithfulness — a weak retriever silently poisons every agent.

3.5 Folder Structure

```
business/
├── process-intelligence/
│   ├── event-logs/
│   ├── discovered-processes/
│   ├── conformance-reports/
│   ├── bottlenecks/
│   └── causal-hypotheses/
├── knowledge-base/
│   ├── rules/
│   ├── decision-patterns/
│   ├── exceptions/
│   ├── best-practices/
│   ├── tacit-knowledge/
│   └── provenance/
├── experts/
│   ├── profiles/
│   ├── shadow-logs/
│   ├── decision-requirement-cards/
│   └── interview-transcripts/
├── materials/
│   ├── documents/
│   ├── regulations/
│   └── sops/
├── distilled/
│   ├── skills/
│   ├── prompts/
│   ├── workflows/
│   ├── checklists/
│   └── playbooks/
├── memory/
│   ├── episodic/
│   ├── semantic/
│   ├── procedural/
│   ├── decision-memory/
│   └── evaluation-memory/
├── evals/
│   ├── golden-tasks/
│   ├── regression-tests/
│   ├── adversarial-tests/
│   ├── human-review-sets/
│   └── benchmark-results/
├── governance/
│   ├── ai-inventory/
│   ├── use-case-risk-tiering/
```

```
│   ├── risk-assessments/
│   ├── human-approval-policy/
│   ├── audit-logs/
│   ├── model-cards/
│   └── assurance-cases/
├── security/
│   ├── threat-models/
│   ├── tool-permissions/
│   ├── prompt-injection-tests/
│   ├── red-team-results/
│   └── incident-reports/
└── evolution/
    ├── workflow-dna/
    ├── successful-variants/
    ├── failed-experiments/
    ├── mutation-history/
    └── lessons-learned/
```

---

4. Execution: Workflow DNA

4.1 Schema

```yaml
workflow_dna:
  id: "wf_customer_onboarding_v12"
  name: "Customer Onboarding"
  domain: "operations"
  objective: "Onboard customer with minimal delay and compliance risk."
  owner: "business_orchestrator"
  version: "12.0"
  inputs: ["signed_contract", "customer_profile", "billing_details"]
  preconditions:
    - "contract_status == signed"
    - "customer_risk_score <= threshold OR legal_approval == true"
  steps:
    - id: "verify_contract"
      agent: "governance_officer"
      tools: ["contract_parser", "policy_retriever"]
    - id: "create_customer_record"
      agent: "business_orchestrator"
      tools: ["crm"]
    - id: "configure_billing"
      agent: "tool_permission_broker"
      tools: ["billing_system"]
    - id: "send_welcome_packet"
      agent: "business_orchestrator"
      tools: ["email"]
  memory_reads: ["contract_rules", "customer_exceptions", "past_failures"]
  memory_writes: ["event_log", "decision_memory", "lessons_learned"]
  guardrails:
    human_approval_required_if:
```

```
        - "risk_tier == high"
        - "contract_exception_detected == true"
        - "tool_action_is_irreversible == true"
    verification:
      required_checks:
        - "crm_record_created"
        - "billing_config_validated"
        - "welcome_packet_sent"
        - "audit_log_complete"
    rollback:
      reversible: true
      rollback_steps: ["disable_customer_record", "void_initial_invoice",
"notify_ops_owner"]
    fitness_metrics:
      - "cycle_time"
      - "error_rate"
      - "customer_satisfaction"
      - "compliance_pass_rate"
      - "human_escalation_rate"
      - "cost_per_case"
```

4.2 Execution Pattern

Use a **bounded state graph**, not a free-form swarm. Reasoning/acting loops (ReAct-style interleaving of thought, action, and observation) are useful *inside* a node, but the graph itself enforces state, permissions, and human-in-the-loop gates.

```
Event → Intake Router → Risk Classifier → Orchestrator
   → [Research] → [Execution] → [Verification] → [Compliance] → [Human Gate]
   → Audit Log + Memory Write → Evaluation → Evolution Sandbox
```

---

5. Evolution Engine (Sandboxed)

5.1 The One Non-Negotiable Rule

> **The Evolution Manager must never mutate production directly.** It may only: propose variants → test in sandbox → compare to baseline → request approval → canary deploy → auto-rollback on failure.

This single constraint converts a risky autonomous swarm into a controlled, auditable system.

5.2 Fitness Function

Do not evolve on subjective preference. Score every variant:

$$F = w_q Q + w_s S + w_c C + w_e E + w_h H - w_r R - w_l L - w_k K$$

Where (Q)=quality, (S)=safety, (C)=compliance, (E)=efficiency, (H)=human satisfaction, (R)=risk penalty, (L)=latency penalty, (K)=cost penalty. For conflicting objectives, use **Pareto selection** rather than collapsing

everything into one scalar.

5.3 Pipeline

```
1.  Observe production / shadow traces
2.  Detect failures, bottlenecks, or opportunities
3.  Generate variants (prompt / workflow / tool-use / role / expert-pattern
crossover)
4.  Test offline against golden tasks
5.  Run security + adversarial tests
6.  Run compliance checks
7.  Replay on historical cases (simulation)
8.  Human review if risk tier requires
9.  Canary deploy to small scope
10. Monitor metrics
11. Promote / rollback / retire
12. Store lessons in evolution memory
```

Natural-language reflection is a powerful optimizer here: reflective prompt-evolution methods (e.g. GEPA) show that a *few* trajectories, diagnosed in language, can beat many rounds of scalar-reward RL — which fits a data-scarce business setting well. Keep these loops **inside the sandbox gates above.**

5.4 Promotion Rule

A variant is promoted only if it (1) improves target metrics, (2) does not regress safety or compliance, (3) passes regression + adversarial tests, (4) has a rollback plan, (5) has complete audit logs, and (6) has human sign-off when the risk tier requires it.

---

6. Governance, Risk & Compliance

Anchor to established frameworks rather than inventing your own.

- **NIST AI RMF (AI 100-1)** — the map/measure/manage risk backbone for trustworthy AI.
- **ISO/IEC 42001** — the management-system wrapper. It is the world's first AI management system standard and is built around the Plan-Do-Check-Act methodology for establishing, implementing, maintaining, and continually improving an AI management system[1], covering risk management, AI system impact assessment, system lifecycle management, and third-party supplier oversight.[3]
- **EU AI Act** — applies if you touch EU users, workers, or regulated decisions. The Act entered into force on 1 August 2024; prohibited practices and AI-literacy obligations applied from 2 February 2025, and GPAI-model obligations from 2 August 2025.[5] Note the moving timeline: Annex III high-risk obligations are being postponed from 2 August 2026 to 2 December 2027[9] via the Digital Omnibus, though this only takes legal effect upon formal adoption and publication. This matters directly here because the Act classifies AI used in employment-related decisions — recruitment, candidate selection, performance evaluation, task allocation, monitoring of workers, and promotion or termination — as high-risk.[2] If the swarm ever touches those, expect requirements around risk management, data governance, technical documentation, record-keeping, transparency, human oversight, accuracy, robustness, and cybersecurity, plus post-market monitoring and incident reporting.[7]

6.1 Autonomy Risk Tiers

Tier	Autonomy	Allowed Behavior
0	Observe	Log and summarize only.
1	Recommend	Suggest; human acts.
2	Draft	Prepare artifacts; human approves before send/execute.
3	Execute (reversible)	Act if rollback exists and risk is low.
4	Execute + gate	Act, but human approves the critical step.
5	Restricted	No autonomous action until an assurance case exists.

6.2 Mandatory Artifacts

AI inventory · use-case risk tiering · human-approval policy · audit logs · incident-response plan · rollback plans · data-retention policy · vendor/model register · tool-permission register · assurance cases · model cards.

---

7. Security

Agentic systems widen the attack surface far beyond classic AppSec. Two OWASP references now apply: the **Top 10 for LLM Applications (2025)** for the model layer and the **Top 10 for Agentic Applications (2026)** for the autonomy layer. The agentic list is a peer-reviewed framework identifying the most critical risks for autonomous AI systems that plan, act, and make decisions across workflows[3], and its highlighted threats include Agent Behavior Hijacking, Tool Misuse and Exploitation, and Identity and Privilege Abuse.[7]

The single most important design fact for a swarm: indirect prompt injection is the key threat in most agentic systems, and after alignment and filtering it should be assumed that prompt injection can still happen — so blast-radius control is critical.[8] Equally, system prompts are not security controls; if a secret is in the prompt, it is already gone.[7] Enforce security deterministically, outside the LLM.

7.1 Control Matrix (mapped to OWASP LLM 2025)

Risk	OWASP	Control
Prompt injection (esp. indirect)	LLM01	Treat all retrieved/user content as untrusted; separate instructions from data; blast-radius limits.
Sensitive info disclosure	LLM02	DLP on outputs/logs; secrets never in prompts; retention limits.
Supply chain	LLM03	Model/tool/adapter registry; provenance; dependency + SBOM scanning.
Data & model poisoning	LLM04	Vet fine-tunes/LoRAs; validate retrieval sources; write-filter memory.
Improper output handling	LLM05	Treat model output as untrusted input; sanitize before any execution.

Risk	OWASP	Control
Excessive agency	LLM06	Risk-tiered autonomy; least privilege; approval gates.
System prompt leakage	LLM07	No secrets/roles in prompts; enforce authz externally.
Vector/embedding weaknesses	LLM08	Tenant isolation on vector stores; poisoned-content detection.
Misinformation	LLM09	Grounding + citations; confidence display; human review on high stakes.
Unbounded consumption	LLM10	Rate limits, timeouts, budget caps ("denial-of-wallet" defense).

For excessive agency specifically, OWASP's practical guidance maps cleanly onto the risk tiers in §6.1: restrict what tools the LLM can access, require human approval for sensitive or irreversible actions, and apply least privilege to data, APIs, and tools.[6]

7.2 Additional Agentic Controls

- **Tool Permission Broker** — narrow, temporary, scoped credentials per task.
- **Memory-poisoning defense** — provenance + human review for high-impact memory writes (a named agentic risk class).
- **Skill/plugin vetting** — third-party agent skills are a live supply-chain vector; scan and pin them.
- **Full observability** — one audit trail across model calls, tool calls, and agent-to-agent traffic.
- **AI incident response** — a defined runbook for GenAI-specific incidents.

---

8. Evaluation System

The biggest gap in most "swarm" designs. Every agent, skill, workflow, and prompt must own:

1. Golden task set 2. Regression tests 3. Adversarial tests 4. Human-review set
2. Historical-replay set 6. Cost/latency benchmark 7. Business-outcome metric 8. Safety/compliance score

Evaluate in realistic, multi-step, tool-using environments — the lesson of agent benchmarks like AgentBench and SWE-bench is that isolated prompt tests do not predict real task performance.

Evaluation card:

```
evaluation:
  target: "wf_customer_onboarding_v12"
  eval_type: "workflow_regression"
  test_set: "historical_onboarding_cases_q2"
  metrics:
    quality_score: 0.94
    compliance_pass_rate: 0.99
    average_cycle_time_minutes: 38
    escalation_rate: 0.12
```

```
      hallucination_rate: 0.01
      unauthorized_tool_attempts: 0
      cost_per_case_usd: 0.42
    result: "pass"
    promotion_decision: "canary_only"
    reviewer: "ops_lead"
```

---

9. Agent Roster

Control / Meta

Agent	Purpose
Business Orchestrator	Routes work, manages state, owns the global objective.
Evolution Manager	Proposes and tests variants (sandbox only).
Evaluation Harness	Runs golden/regression/adversarial/replay suites.
Governance Officer	Applies risk tiers, approval rules, audit requirements.
Security Red-Team	Tests injection, tool misuse, leakage, unsafe autonomy.
Memory Steward	Maintains memory quality, provenance, expiration.
Tool Permission Broker	Grants scoped, temporary tool access.
Incident Commander	Handles failures, rollbacks, postmortems.

Learning

Agent	Purpose
Expert Shadow	Observes experts (with permission).
Cognitive Task Analyst	Turns interviews into decision cards + heuristics.
Process Miner	Discovers workflows from logs.
Task Mining Agent	Observes permitted UI/human task traces.
Conformance Agent	Compares SOPs with observed behavior.
Bottleneck Analyzer	Finds delays, loops, and handoff failures.
Causal Improvement Agent	Proposes sandbox experiments from evidence.
Knowledge Distiller	Converts raw material into rules/skills/playbooks.
Knowledge Curator	Validates, deduplicates, organizes.

---

10. Human-AI Interaction Rules

Synthesizing 20+ years of guidance (Microsoft's Guidelines for Human-AI Interaction), the swarm must: show confidence and uncertainty; explain the evidence used; preview actions before executing them; make correction one click away; allow override; store rejected suggestions as training data; ask for clarification when context is thin; and never hide uncertainty behind confident language.

---

11. 90-Day Rollout

Days 1–14 — Foundation: folder structure · event-log schema · AI inventory · risk tiers · audit logging · first 20 golden tasks.

Days 15–30 — Shadow Learning: enable Shadow Mode · expert interviews · collect event logs · first decision cards · knowledge-ingestion pipeline.

Days 31–60 — Controlled Co-Pilot: RAG + provenance · approval gates · first Workflow DNA · regression tests · Co-Pilot Mode for low/medium-risk workflows.

Days 61–90 — Evolution Sandbox: variant generation · prompt/workflow mutation · eval harness · canary deploy · auto-rollback · promote only variants passing safety, quality, and business tests.

11.1 As-built capability gates (product bar)

The 90-day bands above remain the **vision timeline**. In-repo delivery is tracked as **capability gates** (not calendar days). Product bar mark ~100 maps roughly as:

Phase	structure.md band	As-built (this repo)
A Foundation	Days 1–14	`business/` tree, event schema, inventory, risk tiers, audit, ≥20 golden
B Shadow learning	Days 15–30	Event ingest + PI artifacts; DRC templates + sample cards
C Controlled co-pilot	Days 31–60	Postgres control plane, DNA runs, human gates, Tier-0/1 retrieval, FE ops
D Evolution sandbox	Days 61–90	Corpus eval, canary/rollback, self-improvement (reflect/propose), Improve UI

Evidence: `status.md`, `structure_scorecard_100.md`, `mark_100_verification.md`, `reviews/e1_operator_checklist.md`, `planning/gap_analysis_for_structure.md`.

---

12. Implementation Mapping (SDD + as-built)

12.1 Spec-driven decomposition

Executable sub-functional specs (do not edit as a substitute for this architecture doc):

Path	Contents

Path	Contents
planning/structure/README.md	Index, dependency order, templates
planning/structure/nn_*/requirements.md	EARS requirements
planning/structure/nn_*/design.md	Comprehensive design (v2.0)
planning/structure/nn_*/tasks.md	Implementation tasks (v2.0, marked complete)
planning/structure/DESIGN_QUALITY_SCORE.md	Design portfolio score
planning/structure/TASKS_QUALITY_SCORE.md	Tasks portfolio score
planning/structure/IMPLEMENTATION_STATUS.md	Implement matrix
planning/gap_analysis_for_structure.md	Implement vs tasks gap score

12.2 Section → sub-functional spec

structure.md	Spec folder
§0–1 Charter / principles	01_system-charter-and-design-priorities
§3.5 Folder structure	02_business-artifact-repository
§2 Intake + Risk Router	03_intake-and-risk-router
§6 Governance	04_governance-risk-tiers-and-artifacts
§7 Security	05_security-controls-and-tool-broker
§2.3 Process Intelligence	06_process-intelligence-layer
§3.1–3.2 Elicitation + DRC	07_knowledge-elicitation-and-decision-cards
§3.3 Hybrid memory	08_hybrid-memory-system
§3.4 Tiered retrieval	09_tiered-hybrid-retrieval
§4.1 Workflow DNA	10_workflow-dna-definition
§4.2 Execution pattern	11_bounded-workflow-execution
§4 guardrails + audit path	12_human-gates-and-audit-logging
§8 Evaluation	13_evaluation-harness-and-corpus
§5 Evolution sandbox	14_evolution-sandbox-engine
§9 Agent roster	15_agent-roster-and-control-roles
§10 Human–AI rules	16_human-ai-interaction-rules
§11 Rollout	17_phased-rollout-and-operator-path

12.3 As-built realization notes (does not weaken architecture)

These notes record **how the repo realizes** the architecture today. They do not remove future depth.

Topic	Architecture intent	As-built realization
Harness	Governed agent environment	Dual harness: Trae (`.trae/`) + Grok Build (`.grok/`) via `npm run sync`
Control plane	API + durable state	FastAPI + **Postgres** `runtime_state` JSONB; JSON file = backup/seed only
Tool adapters	Real execution	Local adapters + durable `tool_effects`; live CRM/email = later
Tool broker	Scoped permissions	Allow-list ∩ DNA tools ∩ RBAC ∩ gates; ephemeral OAuth per tool = later
PI agents	Miner / conformance / bottleneck / causal	PI **services + disk artifacts** (not five independent LLM agents)
Retrieval §3.4	Tier 0 / 1 / 2	Tier 0 keyword+hash embed + provenance; Tier 1 entity multi-hop (LightRAG-**lite**); Tier 2 + full LightRAG vendor = later
Knowledge graph	Agent-native graph	K1-lite extract/operators + optional federation export
Evolution §5	Sandbox only	Variants `sandbox_only`; corpus eval; canary; versioned promote; rollback; **no** host code self-rewrite
Self-improvement	Reflective loops (GEPA-style)	Auto-reflect, lessons, auto-propose, Loop runner, FE Improve pipeline
DNA production safety	Gates + rollback	`business:validate` + runtime `activate_workflow_version` production DNA checks
Frontend	Ops console	Next.js live ops (`DEMO_MODE=false`); real forms; Improve; `/app/evolution`
Operator proof	End-to-end path	E1: login → run → human gate → complete → improve (`test_e1_operator_path`)

12.4 Explicit non-goals (current product bar)

Do not treat as missing structure.md requirements for mark ~100:

- Full commercial LightRAG / Neo4j production mesh
- Live external CRM / email / billing SaaS adapters
- DGM-style host application self-rewrite
- Always-on Playwright UI CI with permanent servers
- Infinite enterprise content fill of every `business/` leaf

12.5 Runtime / ops entry points

Layer	Entry

Layer	Entry
Backend	`backend/` — FastAPI, `runtime.py`, infrastructure/*
Frontend	`frontend/` — Next.js ops console
Business corpus	`business/`
Continuity	`memory/handoff.md`, `memory/project.md`, `status.md`

13. References

- NIST AI RMF 1.0 (AI 100-1); ISO/IEC 42001:2023; EU AI Act + Digital Omnibus (2025–2026).
- OWASP Top 10 for LLM Applications (2025); OWASP Top 10 for Agentic Applications (2026); OWASP Agentic Security Initiative.
- Process Mining Manifesto (IEEE Task Force on Process Mining).
- ReAct (Yao et al.); RAG (Lewis et al.); Generative Agents (Park et al.); GEPA (Agrawal et al.); AgentBench (Liu et al.).
- Microsoft, Guidelines for Human-AI Interaction (Amershi et al., CHI 2019); Cognitive Task Analysis / Critical Decision Method (Hoffman, Crandall, Shadbolt).
- In-repo SDD: `planning/structure/`; product evidence: `status.md`, `structure_scorecard_100.md`, `mark_100_verification.md`.